RABBIT HOLES

Accelerate · Manipal University Jaipur

6

PHASE

Forensics

Reading history: finding the commit that broke it, and the person who knows why.

Part of **git started**, Accelerate's third workshop.
github.com/accelerate-muj/git-started · MIT licensed.

Contents

1 History is a database

Most people treat `git log` as a list they scroll past. It is closer to a queryable record of every change anyone made to anything, with timestamps and authorship, and git ships with a genuinely good query language for it.

The situations this phase is for:

- The tests passed last week and fail now, and nobody knows what changed.
- There is a line of code that makes no sense and you need to know why it is there before deleting it.
- A function used to exist and now does not, and you want it back.
- You need to know who to ask.

Every one of these has a direct answer, and most people answer them by guessing.

2 Reading the log properly

The default `git log` is verbose and hard to skim. These are worth learning.

```
git log --oneline --graph --all --decorate
```

The single most useful invocation. `--graph` draws the branch structure in ASCII, `--all` includes every branch rather than just the current one, and `--decorate` labels branches and tags. This is how you see the actual shape of a repository.

2.1 Filtering

Command	Shows
`git log --since="2 weeks ago"`	Recent commits, dates are flexible
`git log --until="2026-08-01"`	Commits before a date
`git log --author="Shashwat"`	One person's commits
`git log --grep="conflict"`	Commits whose *message* matches
`git log -- path/to/file`	Commits that touched one file
`git log --follow -- file`	Same, but across renames
`git log -n 20`	The last twenty
`git log main..feature`	On `feature` but not `main`
`git log feature..main`	On `main` but not `feature`
`git log --merges`	Only merge commits
`git log --no-merges`	Everything except merge commits

The `main..feature` form is worth pausing on. It means "reachable from `feature`, not reachable from `main`", which in practice is exactly "what is in this pull request". Reviewers use it constantly.

2.2 The pickaxe

This is the one almost nobody knows and it is remarkable.

```
git log -S "calculateTotal"
```

This finds every commit that **changed the number of times** that string appears. Not commits mentioning it in the message, and not commits whose diff merely touches a line near it. Commits that added or removed it.

Which means: if a function exists and you want the commit that created it, this finds it in

one command, even if it was created eight years ago and has been moved between four files since.

```
git log -G "def parse_.*line"
```

`-G` is the same idea with a regular expression, matching any diff line that changed.

> **TRY IT**
>
> Something used to work and now does not, and you know a keyword.
>
> ```
> git log -S "MAX_RETRIES" --oneline
> git show <the-sha-that-looks-relevant>
> ```
>
> Two commands, and you are looking at the exact change that introduced or removed the thing. Compare that to scrolling a year of history by hand.

2.3 Custom formats

```
git log --pretty=format:"%h %ad %an %s" --date=short
```

`%h` short hash, `%ad` author date, `%an` author name, `%s` subject. There are around forty of these placeholders in `git help log`.

Save one you like as an alias:

```
git config --global alias.lg "log --graph --oneline --decorate --all"
```

Now `git lg` works everywhere. Phase 11 goes further into aliases.

2.4 Who has been working on what

```
git shortlog -sn
```

Commit counts per author, sorted. Add `--no-merges` for a fairer picture.

> **NOTE**
>
> Commit count is a bad measure of contribution and gets used as one depressingly often. Someone who spends three days finding a one-line fix has done more valuable work than someone who made forty commits renaming variables. Use it to find out *who knows this area*, which is a real question, not to rank people.

3 Diff properly

Command	Compares
`git diff`	Working directory against the index
`git diff --staged`	Index against `HEAD`
`git diff HEAD`	Working directory against `HEAD`
`git diff main feature`	Two branches, tip to tip
`git diff main...feature`	`feature` against the point it diverged
`git diff HEAD~3 HEAD`	Across the last three commits
`git diff --stat`	Summary only, files and line counts
`git diff --word-diff`	Highlights changed words, not whole lines
`git diff --check`	Flags whitespace errors

> **TRAP**
>
> Two dots and three dots mean different things, and the difference matters when you are reviewing.
>
> `git diff main feature` compares the two current tips. If `main` has moved ahead since you branched, the output includes other people's changes as though they were yours.
>
> `git diff main...feature` compares `feature` against the commit where the two branches last agreed. That is what your branch actually changed, and it is what GitHub shows in a pull request.
>
> When reviewing, you almost always want three dots.

`--word-diff` deserves a mention of its own. For prose, which this workshop has plenty of, a normal diff marks the whole line as changed when you altered one word. `--word-diff` shows you the word.

4 Blame

```
git blame sonnet.md
```

Every line, annotated with the commit, author and date that last touched it.

The useful flags:

Flag	Effect
`-L 40,60`	Only lines 40 to 60
`-w`	Ignore whitespace-only changes
`-C`	Detect lines moved from another file
`-M`	Detect lines moved within the file
`--since=1.year`	Stop looking further back

TRAP

Blame's biggest weakness is that it shows the *most recent* change to a line. Someone reformatted the file in 2024, so every line is attributed to them and the actual history is buried.
`-w` and `-M` and `-C` help. So does `.git-blame-ignore-revs`, a file listing commits that blame should look straight through:

```
git config blame.ignoreRevsFile .git-blame-ignore-revs
```

Put the SHA of your big reformat commit in that file, commit it, and blame skips past it forever. GitHub's web blame respects it too.

WHY THIS EXISTS

The command is called blame and the name is unfortunate, because the useful question is never "whose fault is this". It is "what was the author trying to do".
Blame gives you a commit, the commit gives you a message and a pull request, and the pull request gives you the discussion where somebody explained the constraint that makes the strange line necessary. That chain is why commit messages matter. A line of code tells you what, and only the history can tell you why.

5 Bisect

The best tool in git that most people never touch.

You know it worked at some point and is broken now, with two hundred commits in between. Bisect does a binary search: it checks out a commit halfway between good and bad, you say

which it is, and it halves the range again. Two hundred commits takes about eight steps.

```
git bisect start
git bisect bad
git bisect good v1.2.0
```

Git checks out a commit in the middle. Test it, then tell git:

```
git bisect good
```

or

```
git bisect bad
```

Repeat. After a handful of rounds:

```
8e1f4a7c9d2b3a5f6e8d1c4b7a9f2e3d5c6b8a1f is the first bad commit
Author: Someone <someone@example.com>
Date:   Tue Aug 11 14:22:03 2026 +0530

    Refactor the line counter
```

Then:

```
git bisect reset
```

to return to where you started.

5.1 Automatic bisect

This is the part that makes it exceptional. If you can write a command that exits zero for good and non-zero for bad, git will do the whole search unattended.

```
git bisect start HEAD v1.2.0
git bisect run pytest tests/test_parser.py
```

Git checks out commits, runs your tests, interprets the exit code, and prints the guilty commit. You can go and do something else.

Any command works:

```
git bisect run make test
git bisect run ./check.sh
git bisect run grep -q "expected string" output.txt
```

> **NOTE**
>
> Exit code 125 means “cannot test this commit”, and git will skip it rather than counting it as good or bad. Useful when some commits in the range do not build at all. `git bisect skip` does the same thing by hand.

> **DEEPER**
>
> Bisect is the strongest argument for the commit hygiene in Phase 5. Its accuracy is bounded by your commit granularity: if each commit is a self-contained change that builds and passes, bisect lands on the exact line. If your history is forty commits called `wip` and half of them do not compile, bisect gives you a range and a shrug.
> Every tidy commit you make is a future afternoon you do not spend debugging.

6 Finding things that are gone

6.1 Search the current tree

```
git grep "parse_line"
git grep -n "TODO"
git grep -i "shakespeare"
```

Faster than `grep -r` because it only looks at tracked files, skipping `node_modules` and build output automatically.

6.2 Search an old version

```
git grep "parse_line" v1.2.0
git grep "parse_line" HEAD~50
```

6.3 Search every commit ever

```
git grep "the_thing" $(git rev-list --all)
```

Slow on a large repository, and it will find text in a version of a file that was deleted years ago.

6.4 Recover a deleted file

Find the commit that removed it:

```
git log --diff-filter=D --oneline -- path/to/file.py
```

`--diff-filter=D` means “only commits that Deleted this path”. Then bring it back from the commit *before* that one:

```
git checkout <sha>~1 -- path/to/file.py
```

The file is restored and staged. `A` for added, `M` for modified and `R` for renamed work the same way.

7 Reference

Question	Command
What does this repo look like?	`git log --oneline --graph --all`
What is in this branch but not main?	`git log main..feature`
Which commit introduced this string?	`git log -S "string"`
What changed in this PR?	`git diff main...feature`
Who wrote this line and why?	`git blame -w -M file`
Which commit broke it?	`git bisect run <test>`
Where is this text right now?	`git grep "text"`
Which commit deleted this file?	`git log --diff-filter=D -- <path>`
Get a deleted file back	`git checkout <sha>~1 -- <path>`
Who knows this part of the code?	`git shortlog -sn -- <path>`

CHECK YOURSELF

1. What is the difference between `git log --grep` and `git log -S`?
2. Why does a reviewer want `main...feature` rather than `main feature`?
3. Blame says one person wrote every line, because they reformatted the file. What fixes that?
4. Write the two commands that find a build break across 200 commits without you testing anything by hand.
5. A file was deleted six months ago. How do you get it back?

Next: Phase 7, *Merging deeply*, which goes past the conflict you resolved in the workshop into merge strategies, why some merges create a commit and others do not, and how to make git remember a resolution so you never redo it.